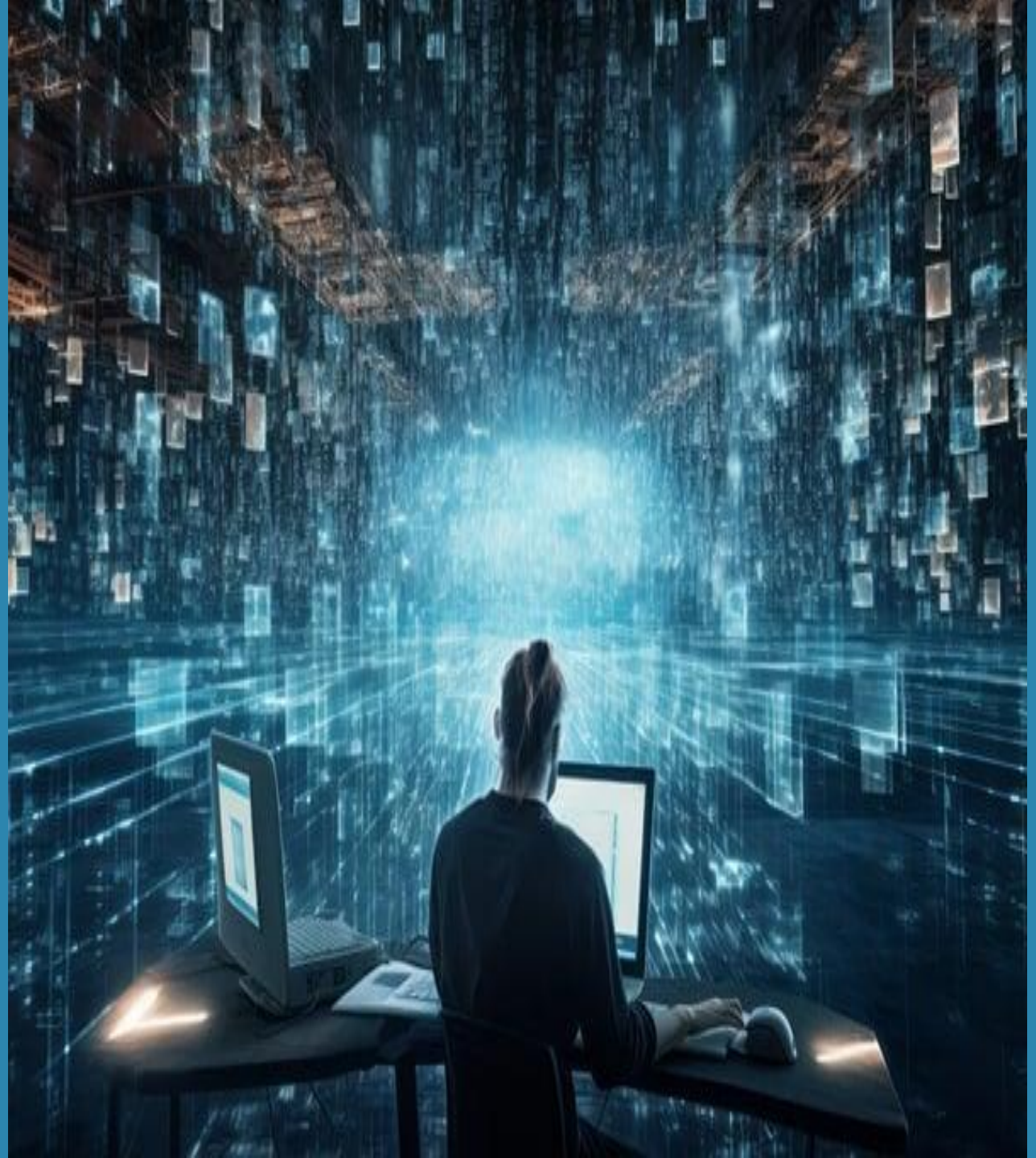


Predicting Corporate Layoffs:

A Comparative Analysis of Private vs. Public Companies

What is Tech Layoffs?

- Tech layoffs refer to the process where technology companies reduce their workforce by terminating employees' contracts. This can happen for various reasons, including: Cost reduction, Strategic shifts, Over-hiring, Economic pressures etc...example: In early 2024, Amazon laid off 27,000 employees, about 8% of its corporate workforce.

Layoffs reasons:

- Cost Reduction: Companies may lay off employees to cut costs and optimize expenses, especially during economic downturns or financial challenges.
- Strategic Shifts: As companies pivot their focus towards new technologies or business priorities (e.g., AI, automation), they may reduce staff in areas that are no longer aligned with their goals.

- Over-Hiring: During periods of rapid growth, companies might hire more employees than needed. When growth stabilizes or slows, they may need to reduce their workforce to balance the payroll.
- Economic Pressures: Broader economic factors, such as recessions or stock market declines, can lead companies to downsize to maintain financial stability.
- Social Contagion: Sometimes, companies follow the trend set by others in the industry, leading to a wave of layoffs across the sector.

Model

What Algorithms can be used to classify tech layoffs statuses?

- Logistic Regression
- Decision Trees
- Random Forest
- KNN
- Naive Bayes ...

Our model :

- In this project, we utilized the Random Forest algorithm due to its robustness and ability to handle various data types effectively. The ensemble nature of Random Forest helps in achieving higher accuracy and stability in predictions, making it a suitable choice for classifying tech layoffs statuses.

```
import panda
data= pd.read_csv(file_path)
df = pd.DataFrame(data)
df.head()
```

Loading...

	company	total_layoffs	impacted_workforce_percentage	reported_date	industry	headquarter_location	sources	status	additional_notes
0	C2FO	20.0	2.0	12/9/2022	Fintech, payments	Leawood, KS	Kansas City Business Journal	Private	NaN
1	VideoAmp	NaN	2.0	9/19/2022	Advertising platforms	Los Angeles	Company memo	Private	NaN
2	Amperity	13.0	3.0	8/20/2022	Saas, analytics	Seattle	Geekwire	Private	NaN
3	Addepar	20.0	3.0	1/18/2023	fintech, data analytics	Mountain View, CA	Fortune	Private	NaN
4	Thirty Madison	24.0	3.0	5/13/2022	Health Care	New York	Axios	Private	Per Axios, 22 of the employees laid off were f...

Dataset Sample

```python
#eda
#Checking for nulls
missing_values = df.isnull().sum()
print(missing_values)
```

```
company                          0
total_layoffs                    0
impacted_workforce_percentage  133
industry                         0
headquarter_location             0
sources                          0
status                           0
dtype: int64
```

Before cleaning the data

After cleaning the data

```python
missing_values = df.isnull().sum()
print(missing_values)
```

```
company                          0
total_layoffs                    0
impacted_workforce_percentage    0
industry                         0
headquarter_location             0
sources                          0
status                           0
dtype: int64
```

Dropping unnecessary Features

```
#clear some unwanted columns
df.drop(columns=['additional_notes'], inplace=True)
df.drop(columns=['reported_date'],inplace = True)
df.head()
```

	company	total_layoffs	impacted_workforce_percentage	industry	headquarter_location	sources	status
0	C2FO	20.0	2.0	Fintech, payments	Leawood, KS	Kansas City Business Journal	Private
1	VideoAmp	NaN	2.0	Advertising platforms	Los Angeles	Company memo	Private
2	Amperity	13.0	3.0	Saas, analytics	Seattle	Geekwire	Private
3	Addepar	20.0	3.0	fintech, data analytics	Mountain View, CA	Fortune	Private
4	Thirty Madison	24.0	3.0	Health Care	New York	Axios	Private

Replacing NaN values with the mean

```
#replace nulls with mean
replace_totLayoffs = df['total_layoffs'].mean()
df['total_layoffs'].fillna(replace_totLayoffs, inplace=True)
df.head()
```

	company	total_layoffs	impacted_workforce_percentage	industry	headquarter_location	sources	status
0	C2FO	20.000000	2.0	Fintech, payments	Leawood, KS	Kansas City Business Journal	Private
1	VideoAmp	450.019544	2.0	Advertising platforms	Los Angeles	Company memo	Private
2	Amperity	13.000000	3.0	Saas, analytics	Seattle	Geekwire	Private
3	Addepar	20.000000	3.0	fintech, data analytics	Mountain View, CA	Fortune	Private
4	Thirty Madison	24.000000	3.0	Health Care	New York	Axios	Private

Encoding results:

df

	company	total_layoffs	impacted_workforce_percentage	industry	headquarter_location	sources	status
0	73	20.000000	2.0	98	51	109	0
1	442	450.019544	2.0	11	56	56	0
2	23	13.000000	3.0	184	113	94	0
3	7	20.000000	3.0	252	70	89	0
4	416	24.000000	3.0	109	76	14	0
...	...	...	...	...	...	...	...
484	361	9090.000000	10.0	251	101	133	1
485	299	450.019544	10.0	181	3	196	1
486	249	450.019544	10.0	51	76	118	1
487	375	450.019544	10.0	150	101	174	1
488	60	450.019544	10.0	81	101	119	0

```python
#replace nulls with mode
replace_perc=df['impacted_workforce_percentage'].mode()[0]
df['impacted_workforce_percentage'].fillna(replace_perc, inplace=True)
df.head()
```

```python
#encoding company column using label encoding
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['company'] = label_encoder.fit_transform(df['company'])
df.head()
```

```python
#encoding sources column using label encoding
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['sources'] = label_encoder.fit_transform(df['sources'])
df.head()
```

```python
#encoding headquarter locations column using label encoding
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['headquarter_location'] = label_encoder.fit_transform(df['headquarter_location'])
df.head()
```

```python
#encoding industry column using label encoding
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['industry'] = label_encoder.fit_transform(df['industry'])
df.head()
```

```python
#encoding status column using label encoding
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['status'] = label_encoder.fit_transform(df['status'])
df.head()
```

Training the data

Introduce The Model:

RandomForestClassifier

```
#train test split
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
data_x = df.iloc[:,:-1].values
data_y = df.iloc[:, -1].values
X_train, X_test, y_train, y_test = train_test_split(data_x, data_y, test_size = 0.2)
model = RandomForestClassifier()
model.fit(X_train, y_train)
y_predict = model.predict(X_test)
print(y_predict)
```

```
[0 1 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1 0
 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 1 0 0]
```

Performance Metrics

```python
#performance metrics
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

print(confusion_matrix(y_test,y_predict))
print(accuracy_score(y_test,y_predict))
print(classification_report(y_test,y_predict))
```

```
[[67  3]
 [17 11]]
0.7959183673469388
              precision    recall  f1-score   support

           0       0.80      0.96      0.87        70
           1       0.79      0.39      0.52        28

    accuracy                           0.80        98
   macro avg       0.79      0.68      0.70        98
weighted avg       0.79      0.80      0.77        98
```

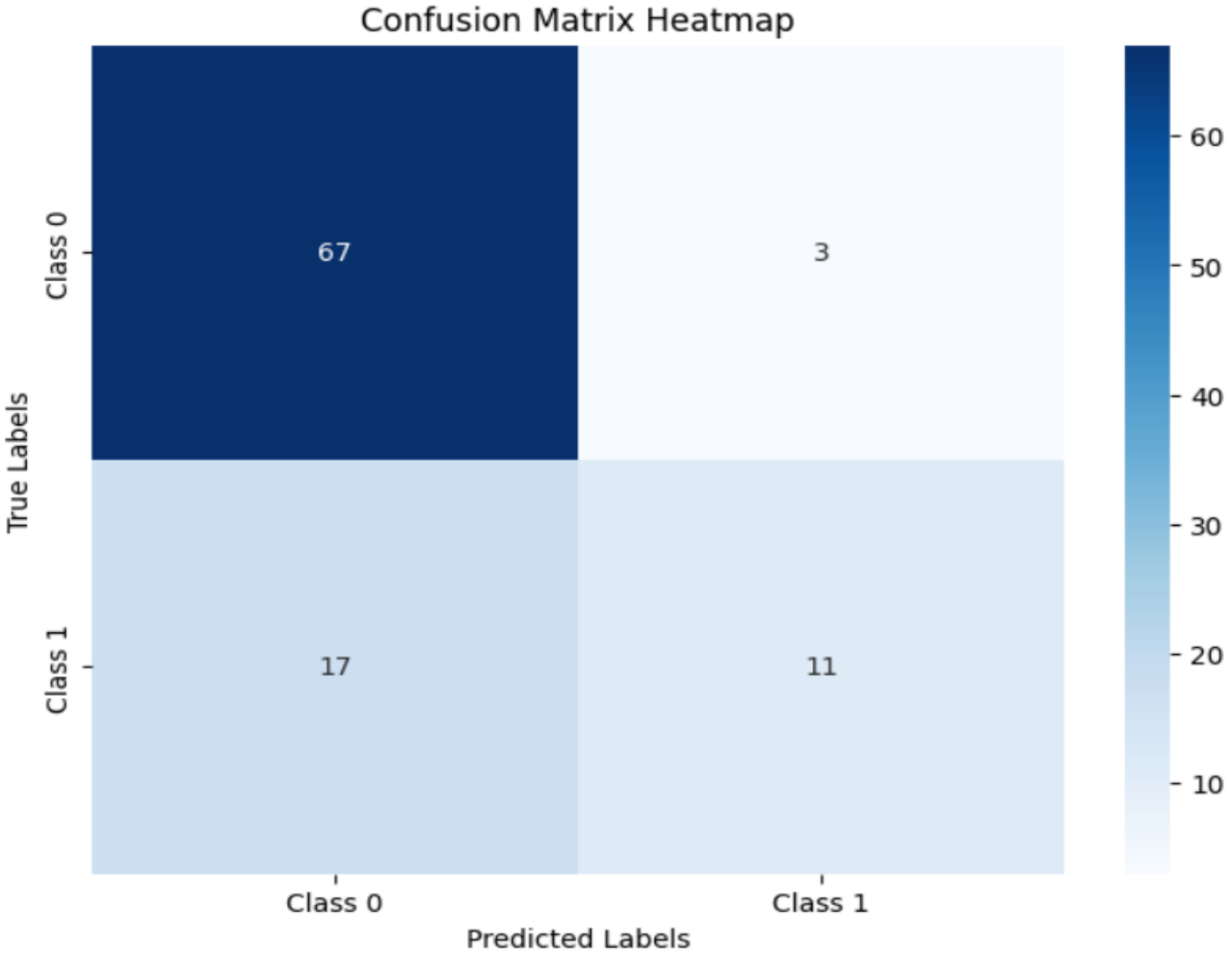
Features Correlation

```
df.corr()
```

	company	total_layoffs	impacted_workforce_percentage	industry	headquarter_location	sources	status
company	1.000000	-0.062616	0.024405	0.025848	0.035266	0.115711	0.067007
total_layoffs	-0.062616	1.000000	-0.012610	-0.029916	0.060215	0.035247	0.236688
impacted_workforce_percentage	0.024405	-0.012610	1.000000	-0.062117	0.022367	0.056719	-0.141566
industry	0.025848	-0.029916	-0.062117	1.000000	-0.064092	0.061575	-0.008251
headquarter_location	0.035266	0.060215	0.022367	-0.064092	1.000000	0.140164	0.059117
sources	0.115711	0.035247	0.056719	0.061575	0.140164	1.000000	-0.006864
status	0.067007	0.236688	-0.141566	-0.008251	0.059117	-0.006864	1.000000

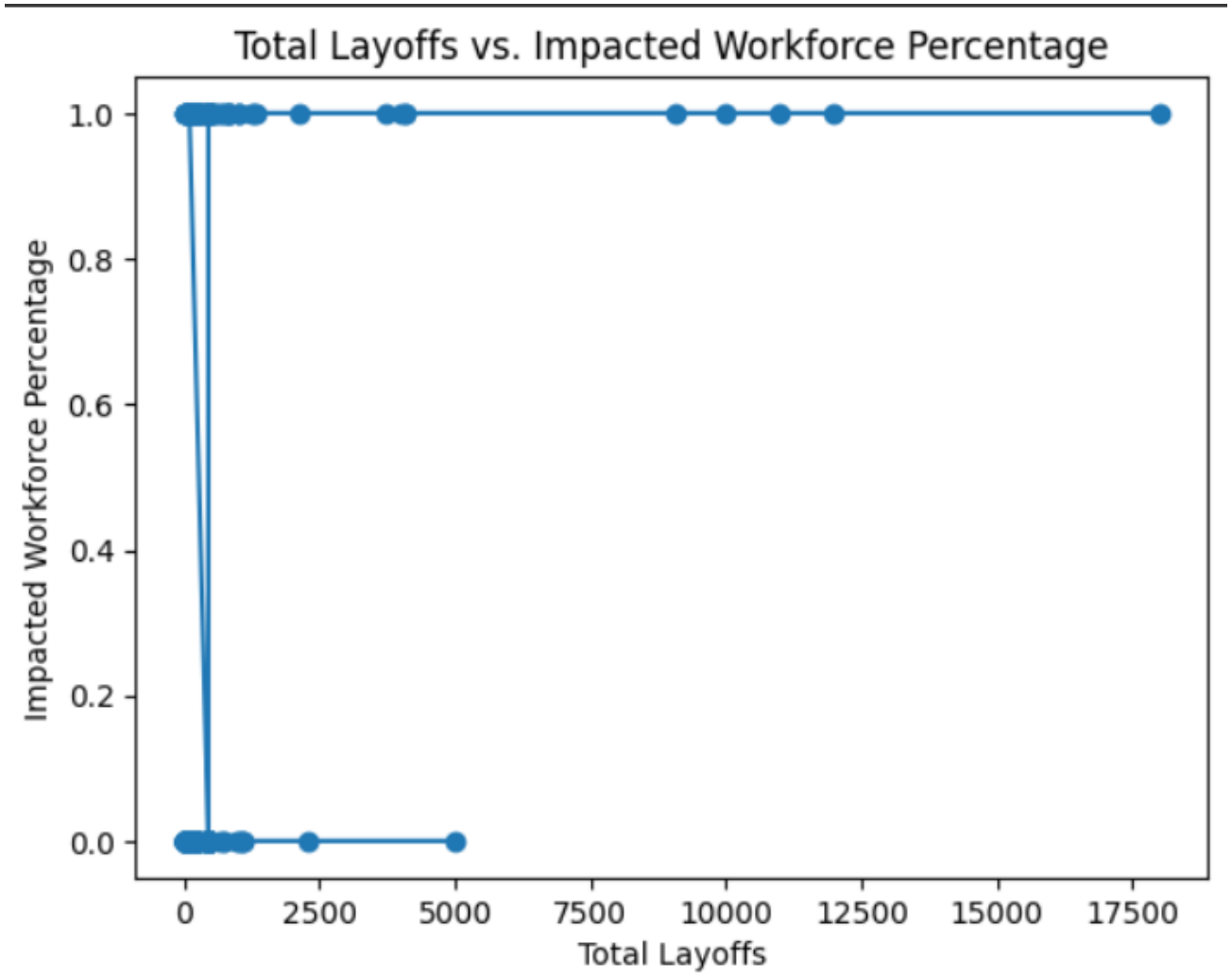
Confusion Matrix

- import seaborn as sns
- import matplotlib.pyplot as plt
- cm = confusion_matrix(y_test, y_predict)
- plt.figure(figsize=(8, 6))
- sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Class 0', 'Class 1'], yticklabels=['Class 0', 'Class 1'])
- plt.title('Confusion Matrix Heatmap')
- plt.xlabel('Predicted Labels')
- plt.ylabel('True Labels')
- plt.show()



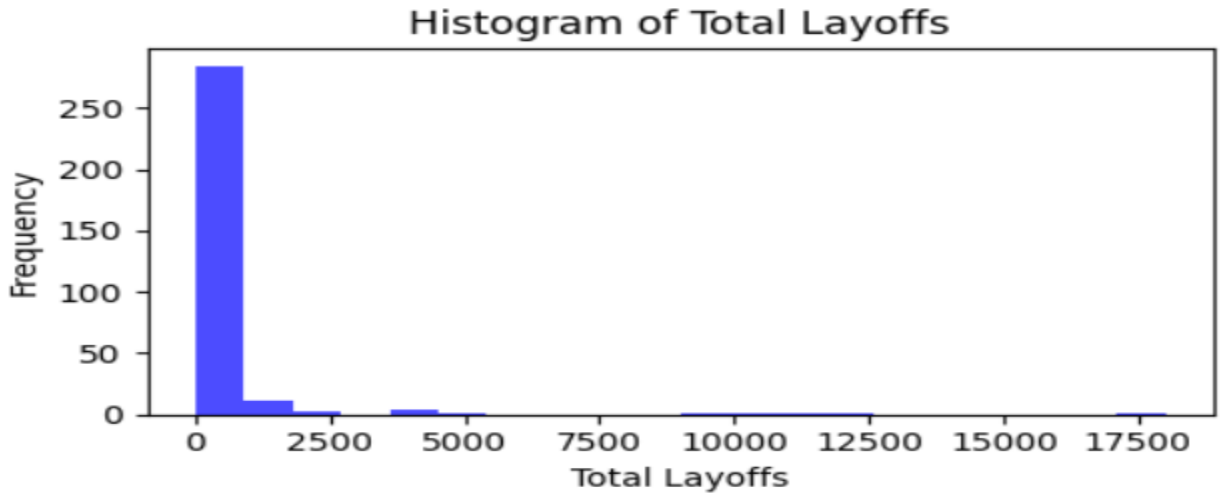
Relation between some Features

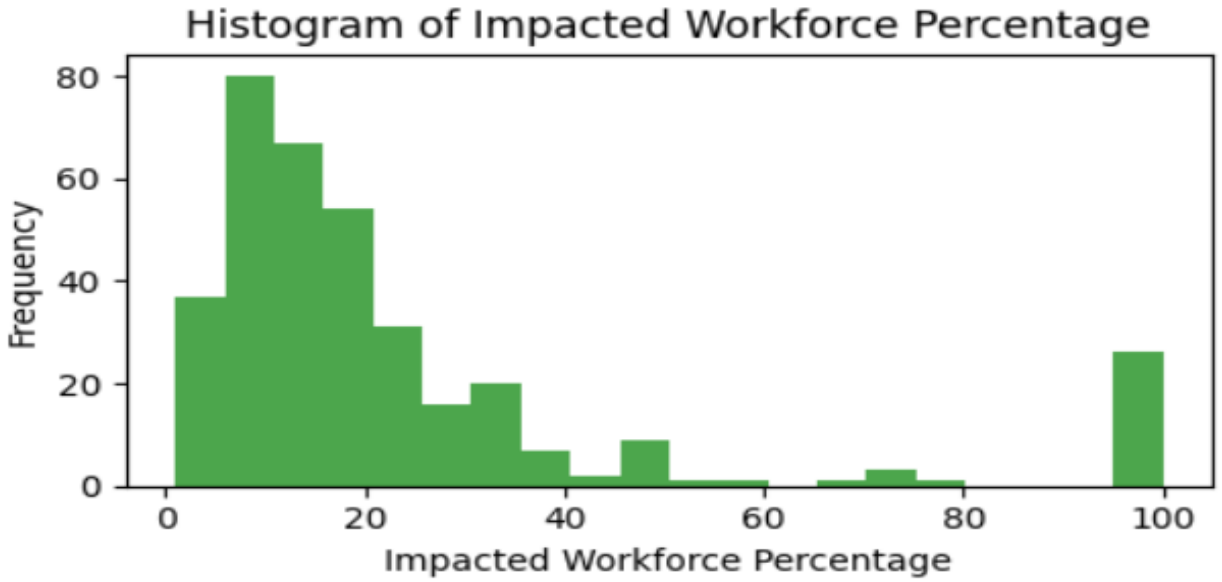
- import matplotlib.pyplot as plt
- plt.plot(df['total_layoffs'], df['status'], marker='o')
- plt.xlabel('Total Layoffs')
- plt.ylabel('Impacted Workforce Percentage')
- plt.title('Total Layoffs vs. Impacted Workforce Percentage')
- plt.show()



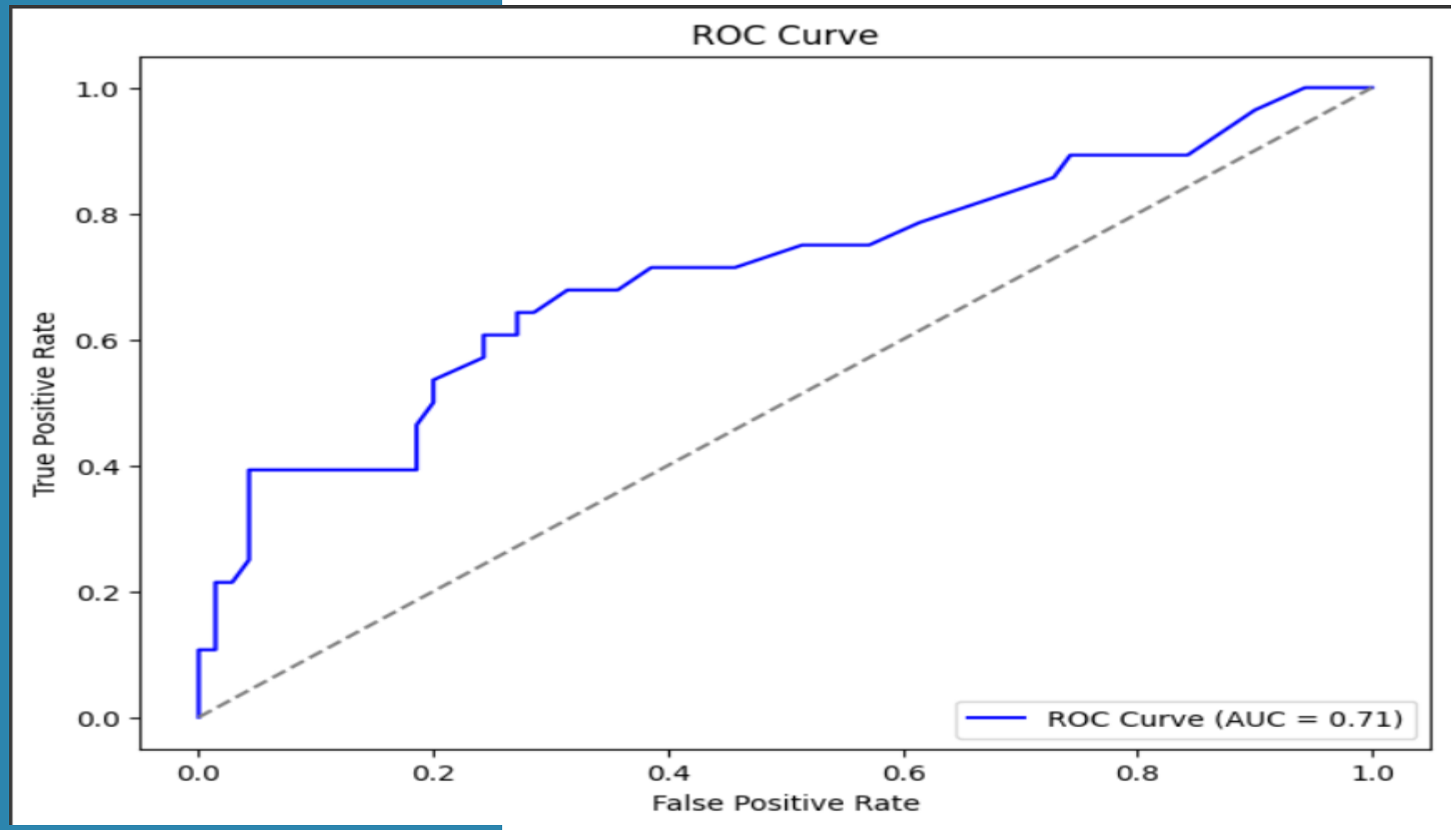
Visualization for some Features

- import matplotlib.pyplot as plt
- plt.figure(figsize=(10, 3))
- plt.subplot(1, 2, 1)
- plt.hist(df['total_layoffs'], bins=20, alpha=0.7, color='blue')
- plt.xlabel('Total Layoffs')
- plt.ylabel('Frequency')
- plt.title('Histogram of Total Layoffs')
- plt.subplot(1, 2, 2)
- plt.hist(df['impacted_workforce_percentage'], bins=20, alpha=0.7, color='green')
- plt.xlabel('Impacted Workforce Percentage')
- plt.ylabel('Frequency')
- plt.title('Histogram of Impacted Workforce Percentage')
- plt.tight_layout()
- plt.show()

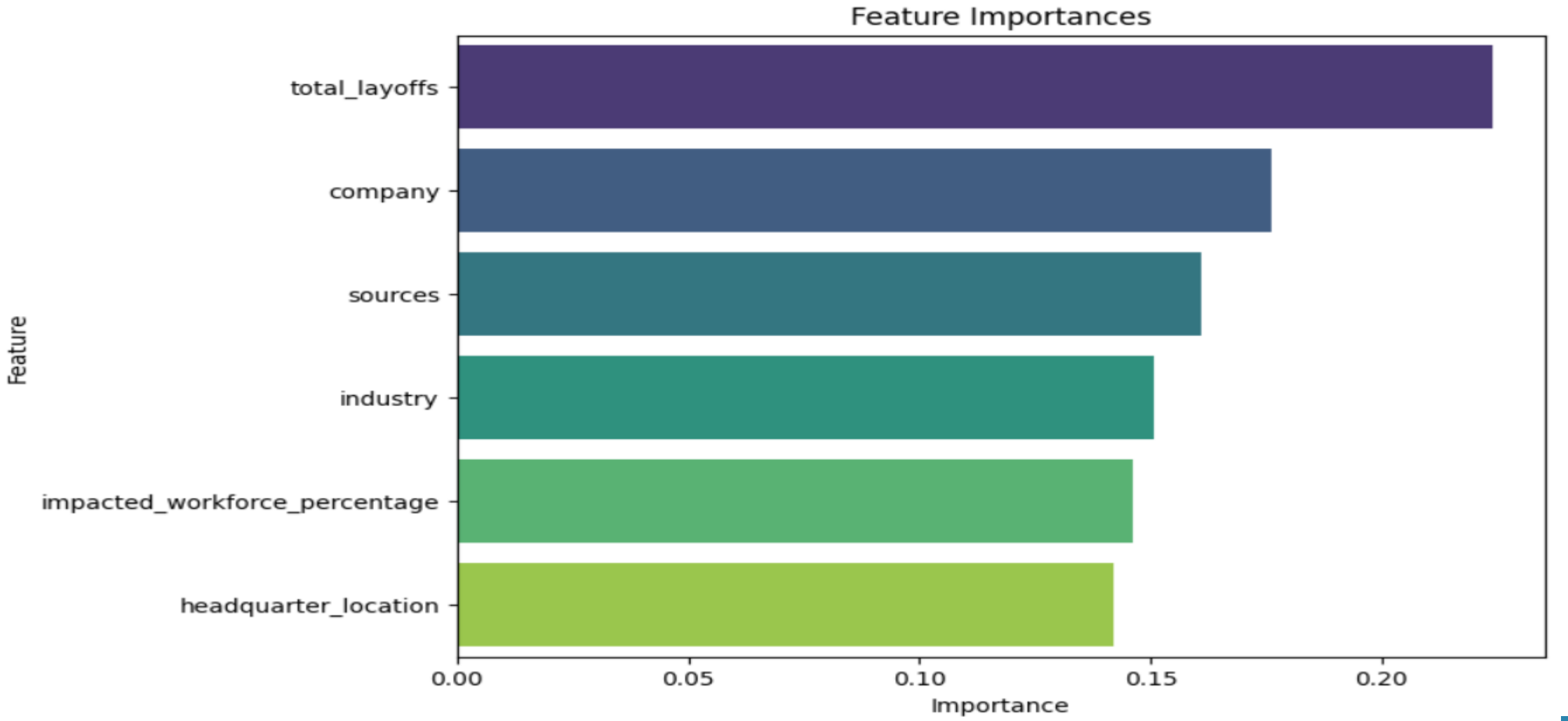




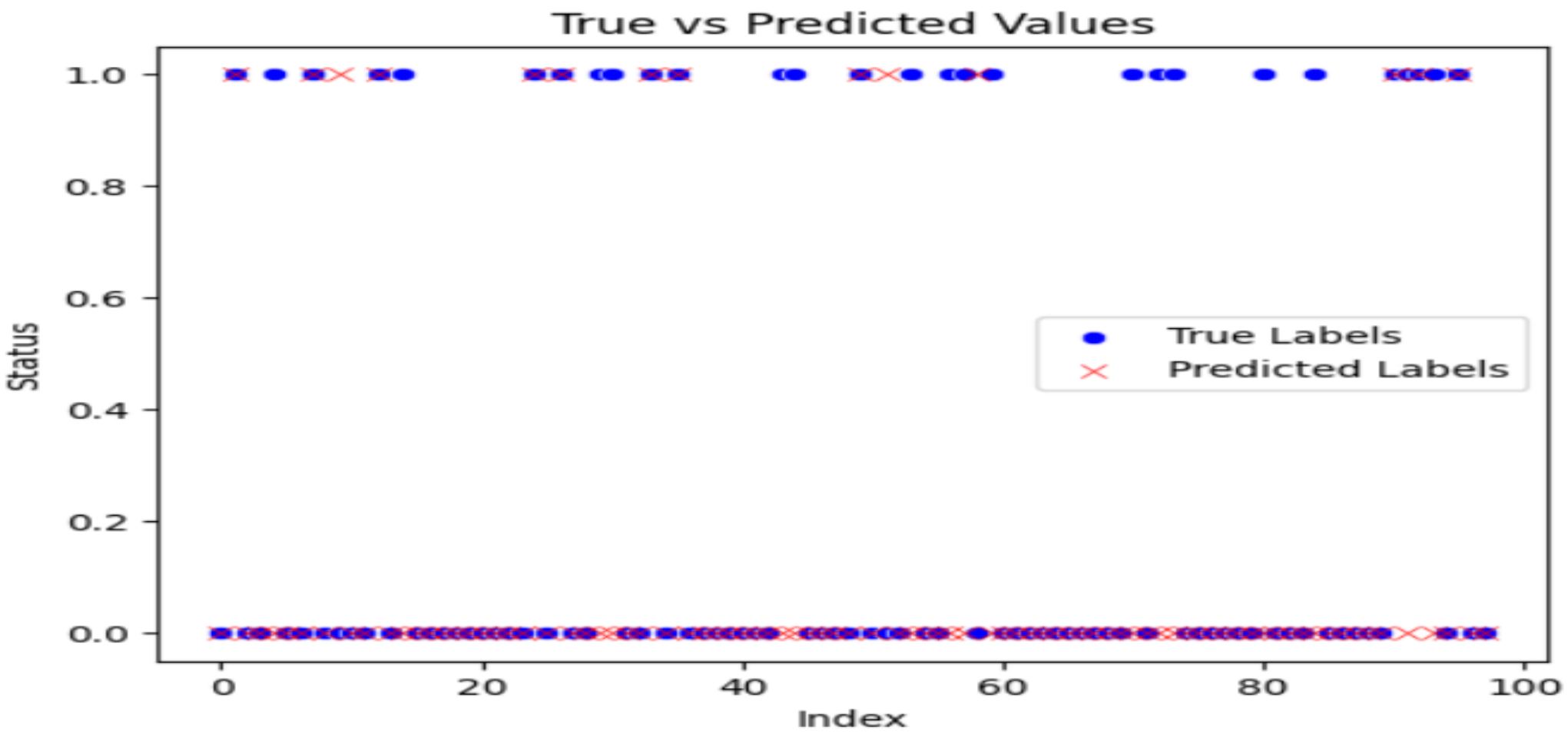
Receiver Operating Characteristic



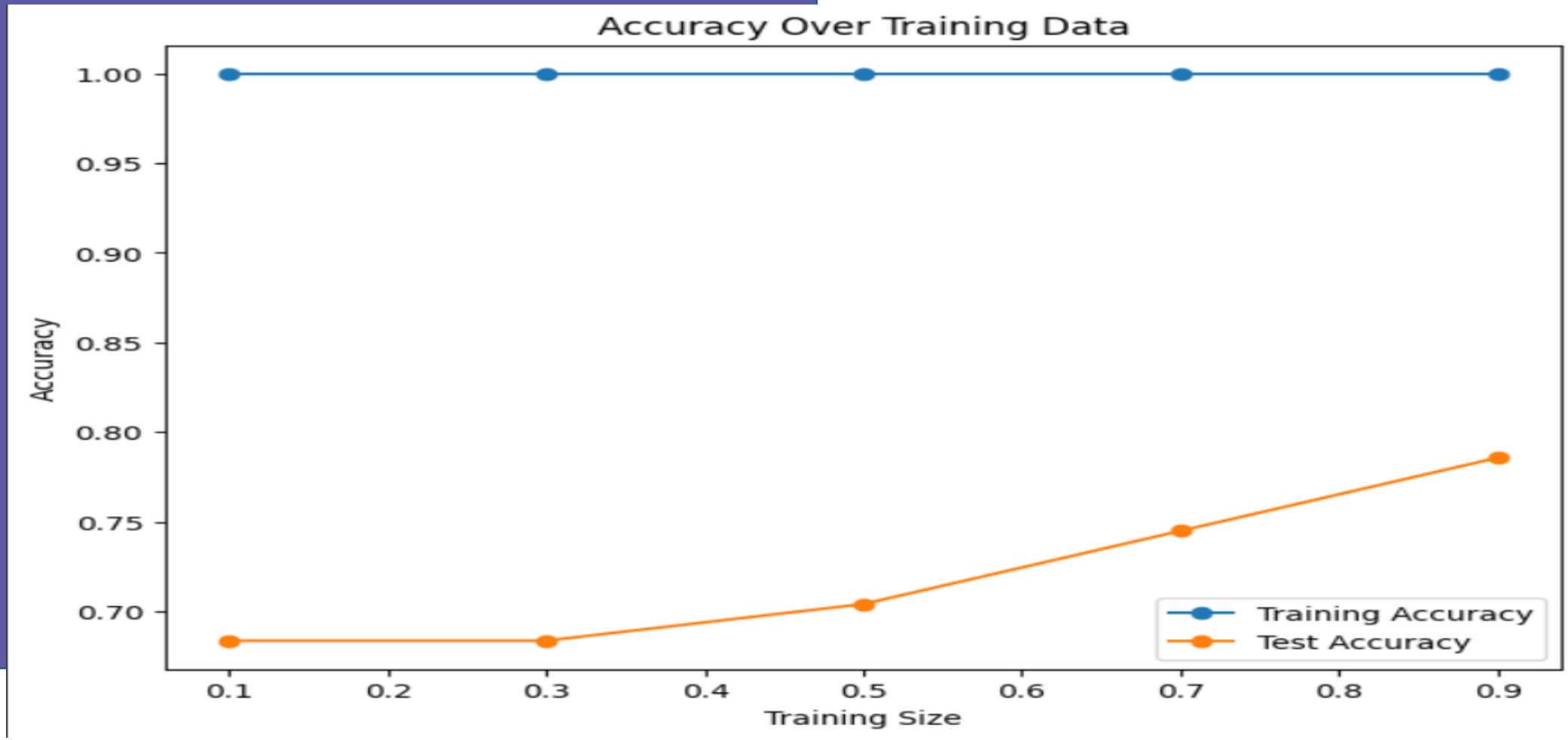
Feature Importance



Predicted vs True Values



Accuracy over Training Data



Thank you for Listening

☺☺

Presented By :

Jennifer Mikaelian

Elias Bou Moussa

Edward Habra